计算机学院实验报告

<table>
<tr><td colspan="2">实验题目：光线追踪渲染图像</td><td>学号：202200130077</td></tr>
<tr><td>日期:2024. 5 月 28 日</td><td>班级：智能</td><td>姓名：薛润泽</td></tr>
<tr><td colspan="3">Email：</td></tr>
<tr><td colspan="3">实验目的：
1. 从 main 函数开始。我们定义场景的参数，添加物体（球体或三角形）到场景
中，并设置其材质，然后将光源添加到场景中。
2. 调用 Render(scene) 函数。在遍历所有像素的循环里，生成对应的光线并将返回的颜色保存在帧缓冲区（framebuffer）中。在渲染过程结束后，帧缓冲区中的信息将被保存为图像。
3. 在生成像素对应的光线后，我们调用 CastRay 函数，该函数调用 trace 来查询光线与场景中最近的对象的交点。
4. 然后，我们在此交点执行着色。我们设置了三种不同的着色情况，并且已经为你提供了代码。
你需要修改的函数是：
• Renderer.cpp 中的 Render()：这里你需要为每个像素生成一条对应的光线，然后调用函数 castRay()来得到颜色，最后将颜色存储在帧缓冲区的相应像素中。
• Triangle.hpp 中的 rayTriangleIntersect(): v0, v1, v2 是三角形的三个顶点，orig 是光线的起点，dir 是光线单位化的方向向量。tnear, u, v 是你需要使用我们课上推导的 Moller-Trumbore 算法来更新的参数。</td></tr>
<tr><td colspan="3">实验环境介绍：
Ubuntu</td></tr>
</table>

解决问题的主要思路：

1、Recursive (Whitted-Style) Ray Tracing：光线追踪算法，主要原理是使用了光路可逆原理。物品反射到视角中的光线可以等价为视角射向物品的光线。主要思路为从视角中射向对应屏幕像素点一道光线，穿过像素点后，光线会与空间中的物体相碰撞，并且由物体反射，与物体碰撞后可以获得碰撞点处物体的颜色信息，然后每个碰撞点颜色信息都可以返回到该像素点，做累加后就完成了光线追踪。

2、Möller Trumbore Algorithm：计算与三角形求交的算法，使用三角形重心坐标公式并使用矩阵计算，算得使用三角形重心坐标 b1,b2 表示的交点和 O 点处所加的单位向量长度 t，通过判断 t，b1，b2 的大小来判断交点在三角形内还是三角形外。

$$\vec{\mathbf{O}} + t\vec{\mathbf{D}} = (1 - b_1 - b_2)\vec{\mathbf{P}}_0 + b_1\vec{\mathbf{P}}_1 + b_2\vec{\mathbf{P}}_2$$

$$\begin{bmatrix} t \\ b_1 \\ b_2 \end{bmatrix} = \frac{1}{\vec{\mathbf{S}}_1 \cdot \vec{\mathbf{E}}_1} \begin{bmatrix} \vec{\mathbf{S}}_2 \cdot \vec{\mathbf{E}}_2 \\ \vec{\mathbf{S}}_1 \cdot \vec{\mathbf{S}} \\ \vec{\mathbf{S}}_2 \cdot \vec{\mathbf{D}} \end{bmatrix}$$

Cost = (1 div, 27 mul, 17 add)

Where:

$$\vec{\mathbf{E}}_1 = \vec{\mathbf{P}}_1 - \vec{\mathbf{P}}_0$$
$$\vec{\mathbf{E}}_2 = \vec{\mathbf{P}}_2 - \vec{\mathbf{P}}_0$$
$$\vec{\mathbf{S}} = \vec{\mathbf{O}} - \vec{\mathbf{P}}_0$$
$$\vec{\mathbf{S}}_1 = \vec{\mathbf{D}} \times \vec{\mathbf{E}}_2$$
$$\vec{\mathbf{S}}_2 = \vec{\mathbf{S}} \times \vec{\mathbf{E}}_1$$

Recall: How to determine if the “intersection” is inside the triangle?

Hint:
(1-b1-b2), b1, b2 are barycentric coordinates!

本实验的主要思路如下：

1. 在 Reader()函数中为视角生成一条主光线，该光线穿过了对应遍历的像素点，并与空间中物体发生相交，相交得到的颜色值存储在数组中。
2. 在 Triangle.hpp 中实现 Möller Trumbore Algorithm 算法判断光线是否与该三角形相交，具体变量定义如上图所示。

实验步骤与实验结果：

1、Recursive (Whitted-Style) Ray Tracing

从视角发出一段光线，该光线与物体空间中的物体进行多次相交（因为光线会被物体反射，所以可以多次），将交点处的颜色信息保存到一个数组中，然后返回给对应像素点。

代码如下：

```
void Renderer::Render(const Scene& scene)
{
    std::vector<Vector3f> framebuffer(scene.width *
scene.height);

    float scale = std::tan(deg2rad(scene.fov * 0.5f));
```

```
    float imageAspectRatio = scene.width /
(float)scene.height;

    // Use this variable as the eye position to start your
rays.
    Vector3f eye_pos(0);
    int m = 0;
    for (int j = 0; j < scene.height; ++j)
    {
        for (int i = 0; i < scene.width; ++i)
        {
            // generate primary ray direction
            float x;
            float y;
            // TODO: Find the x and y positions of the current
pixel to get the direction
            // vector that passes through it.
            // Also, don't forget to multiply both of them with
the variable *scale*, and
            // x (horizontal) variable with the
*imageAspectRatio*
            x = i + 0.5; y = j + 0.5;
            x = 2 * x / scene.width - 1; y = 2 * y /
scene.height - 1;
            x = x * imageAspectRatio * scale; y = (-1) * y *
scale;
            Vector3f dir = normalize(Vector3f(x, y, -1)); //
Don't forget to normalize this direction!
            framebuffer[m++] = castRay(eye_pos, dir, scene, 0);
        }
        UpdateProgress(j / (float)scene.height);
    }

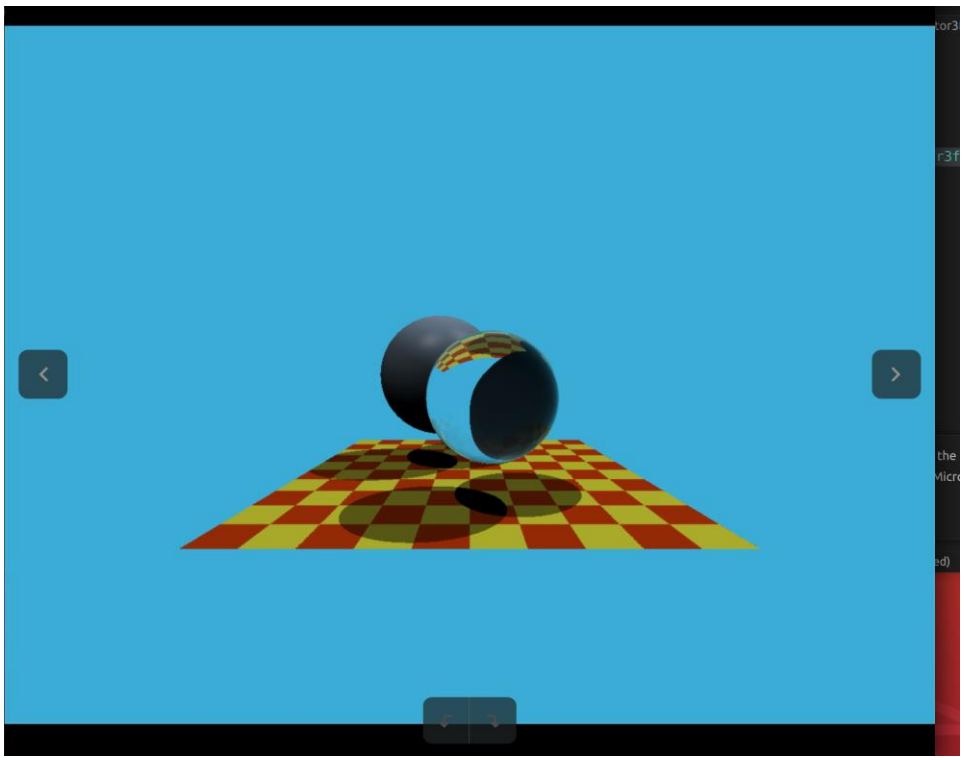
    // save framebuffer to file
    FILE* fp = fopen("binary.ppm", "wb");
    (void)fprintf(fp, "P6\n%d %d\n255\n", scene.width,
scene.height);
    for (auto i = 0; i < scene.height * scene.width; ++i) {
        static unsigned char color[3];
        color[0] = (char)(255 * clamp(0, 1, framebuffer[i].x));
        color[1] = (char)(255 * clamp(0, 1, framebuffer[i].y));
        color[2] = (char)(255 * clamp(0, 1, framebuffer[i].z));
        fwrite(color, 1, 3, fp);
    }
```

2、Möller Trumbore Algorithm：

计算与三角形是否有交点，具体代码如下：

```
bool rayTriangleIntersect(const Vector3f& v0, const Vector3f&
v1, const Vector3f& v2, const Vector3f& orig,
                          const Vector3f& dir, float& tnear, float&
u, float& v)
{
    // TODO: Implement this function that tests whether the
triangle
    // that's specified bt v0, v1 and v2 intersects with the ray
(whose
    // origin is *orig* and direction is *dir*)
    // Also don't forget to update tnear, u and v.
    auto E1 = v1 - v0;
    auto E2 = v2 - v0;
    auto S = orig - v0;
    auto S1 = crossProduct(dir, E2);
    auto S2 = crossProduct(S, E1);
    auto under = dotProduct(S1, E1);
    if(abs(under) <= 0.00001) return false;
    auto t = 1.0 * dotProduct(S2,E2) / under;
    auto b1 = 1.0 * dotProduct(S1, S) / under;
    auto b2 = 1.0 * dotProduct(S2, dir) / under;
    if(t >= 0 && b1 >= 0 && b2 >= 0 && 1 - b1 - b2 >= 0){
        tnear = t;
        u = b1;
        v = b2;
        return true;
    }
    return false;
}
```

结果：



实验中存在的问题及解决：

问题 1： 在 render()函数中，如果将的更新写为 y = 2 * y / scene.height - 1;会发生什么情况？

答：最后生成的图像会上下颠倒

问题 2：玻璃球能够显示出后方的黑球和下方的彩色块，最少需要几次光线反射？
答：5 次